

Working with PDFs in Python is straightforward using the modern **pypdf** library. Older tutorials often mention PyPDF2, but that library was deprecated and merged back into pypdf, which is actively maintained, faster, and much less likely to corrupt complex layouts.

Prerequisites

You will need to install the library first. Open your terminal or command prompt and run:

```
pip install pypdf
```

1. The PDF Merger Code

This script takes a list of PDF files and stitches them together sequentially into a single document.

```
import os
from pypdf import PdfMerger

def merge_pdfs(pdf_list, output_filename="merged_output.pdf"):
    """
    Merges multiple PDF files into a single output PDF document.
    """
    merger = PdfMerger()

    print("Starting PDF merge process...")

    try:
        for pdf in pdf_list:
            if not os.path.exists(pdf):
                print(f" Warning: File '{pdf}' not found. Skipping.")
                continue

            print(f"Appending: {pdf}")
            # Append the entire PDF file into our merger engine
instance      merger.append(pdf)

            # Write the combined file stream down to your drive disk space
            merger.write(output_filename)
            print(f"\n Success! All PDFs merged into: {output_filename}")

    except Exception as e:
        print(f" An error occurred during merging: {e}")
    finally:
        # Always close the merger object to clean up memory handles
        merger.close()

if __name__ == "__main__":
    # Define files you want to merge in sequential order
```

```

files_to_combine = ["document1.pdf", "document2.pdf",
"document3.pdf"]

# Run the merger tool
merge_pdfs(files_to_combine, "final_combined_report.pdf")

```

2. The PDF Splitter Code

This script takes a single PDF document and breaks it apart. It can extract a single page, a specific range, or split the entire file into individual single-page documents.

```

import os
from pypdf import PdfReader, PdfWriter

def split_pdf_by_range(input_pdf, start_page, end_page,
output_filename="split_output.pdf"):
    """
    Extracts a specific range of pages from a PDF.
    Note: pypdf uses 0-indexed pages (Page 1 is index 0).
    """
    if not os.path.exists(input_pdf):
        print(f"Error: File '{input_pdf}' does not exist.")
        return

    # Initialize the reader and writer modules
    reader = PdfReader(input_pdf)
    writer = PdfWriter()

    # Get total pages available inside the document
    total_pages = len(reader.pages)
    print(f"Analyzing '{input_pdf}' ({total_pages} total pages)...")

    # Adjust human numbers (1-indexed) to match Python's tracking
    arrays (0-indexed)
    start_index = start_page - 1
    end_index = min(end_page, total_pages) # Safe fallback bound guard

    print(f"Extracting pages {start_page} to {end_index}...")

    # Loop through the desired page window space range
    for page_num in range(start_index, end_index):
        # Target the page object and append it straight to our writer
        array stack
        writer.add_page(reader.pages[page_num])

    try:
        # Write out our extracted page blocks to a fresh target output
        file

```

```

        with open(output_filename, "wb") as output_file:
            writer.write(output_file)
        print(f" Success! Extracted range saved as:
{output_filename}")
    except Exception as e:
        print(f" Failed to write out split file. Error: {e}")

if __name__ == "__main__":
    # Example: Extract pages 2 through 5 from a large document file
    split_pdf_by_range(
        input_pdf="large_report.pdf",
        start_page=2,
        end_page=5,
        output_filename="extracted_section.pdf"
    )

```

Workflow Framework

1. Install Dependency

Terminal Command

Install the modern utility codebase via pip:

2. Prepare Your Files

Script Design

Place the PDF files you want to manipulate in the same directory where your script runs, or use absolute file paths (like `r"C:\Users\Name\Documents\file.pdf"`).

3. Execute Code

Deployment

Save the code blocks as separate files (`merge.py` and `split.py`) and execute them via terminal to handle your document processing tasks cleanly.